



CS4051 Fundamentals of Computing

**70% Individual Coursework**

**Autumn 2024/25**

**Student Name:** Shrawan karki

**London Met ID:** 25028798

**College ID:** NP05CP4A250078

**Assignment Due Date:** Friday, May 8, 2026

**Assignment Submission Date:** Friday, May 8, 2026

GO to: [Link](#)

*I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.*

# Contents

|                                                  |    |
|--------------------------------------------------|----|
| 1.Introduction .....                             | 1  |
| 1.1    Project overview.....                     | 1  |
| 1.2    Main Purpose.....                         | 1  |
| 1.3    Aims and Objectives .....                 | 2  |
| 1.4    Technologies Used.....                    | 3  |
| 1.4.1    Python .....                            | 3  |
| 1.4.2    VS code.....                            | 4  |
| 2    Algorithm.....                              | 5  |
| 2.1    Algorithm 1: Whole program base.....      | 5  |
| 2.2    Algorithm 2: Module based algorithm.....  | 7  |
| 2.2.1    Algorithm of choices.py .....           | 7  |
| 2.2.2    Algorithm of display.py.....            | 7  |
| 2.2.3    Algorithm of sales.py .....             | 7  |
| 2.2.4    Algorithm of restock.py .....           | 8  |
| 2.2.5    Algorithm of main.py.....               | 9  |
| 3    Pseudocode.....                             | 10 |
| 3.1    Pseudocode for sales.py.....              | 10 |
| 3.2    Pseudocode for Restock.py .....           | 12 |
| 4    Flowchart.....                              | 13 |
| 4.1    Sale Module Flowchart .....               | 14 |
| 4.2    Restock Module Flowchart.....             | 15 |
| 5    Data Structures.....                        | 16 |
| 5.1    Types of Data Structures .....            | 16 |
| 5.1.1    Primitive Data Structures.....          | 16 |
| 5.1.2    Collection Data Structures.....         | 17 |
| 5.2    Data Structures Used in This Program..... | 18 |
| 5.3    Explanation of Each Data Structure .....  | 18 |
| 5.3.1    List.....                               | 19 |
| 5.3.2    Dictionary .....                        | 20 |
| 5.3.3    String.....                             | 21 |
| 6    Program.....                                | 22 |

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
| 6.1  | Program explanation.....                                          | 22 |
| 6.2  | Source Code.....                                                  | 22 |
| 6.3  | Exception Handling.....                                           | 22 |
| 6.4  | User Interface .....                                              | 24 |
| 7    | Testing .....                                                     | 25 |
| 7.1  | Test Case 1: View medicine from inventory .....                   | 25 |
| 7.2  | Test Case 2: Main menu Validation .....                           | 26 |
| 7.3  | Test Case 3: Exception Handling (Invalid Input).....              | 27 |
| 7.4  | Test Case 4: Stock Validity check .....                           | 28 |
| 7.5  | Test Case 5: Multi item Processing and Discount Calculation ..... | 29 |
| 7.6  | Test Case 6: Automatic invoice generator .....                    | 31 |
| 7.7  | Test Case 7: Medicine range and Quantity range validation .....   | 32 |
| 7.8  | Test Case 8: Unique Restock Note Generation.....                  | 33 |
| 7.9  | Test Case 9: Dynamic file update inventory.txt.....               | 34 |
| 7.10 | Test Case 10: Inventory Accuracy .....                            | 35 |
| 8    | Conclusion .....                                                  | 37 |
| 9    | Appendix.....                                                     | 38 |
| 9.1  | Main.py .....                                                     | 38 |
| 9.2  | Restock.py .....                                                  | 40 |
| 9.3  | Sales.py .....                                                    | 43 |
| 9.4  | Display.py.....                                                   | 46 |
| 9.5  | Choice.py.....                                                    | 47 |
| 9.6  | Inventory.txt.....                                                | 48 |

## Table of Tables

|                                                      |    |
|------------------------------------------------------|----|
| Table 1 : Primitive Data Structures .....            | 16 |
| Table 2 : Collection Data Structures .....           | 17 |
| Table 3 : Data Structures Used in This Program ..... | 18 |
| Table 4 : Test case 1 table .....                    | 25 |
| Table 5 :Test case 2 table .....                     | 26 |
| Table 6 : Test case 3.....                           | 27 |
| Table 7 : Test case 4 table .....                    | 28 |
| Table 8: Test case 5.....                            | 29 |
| Table 9 : Test case 6 table .....                    | 31 |
| Table 10 : Test case 7 .....                         | 32 |
| Table 11 : Test case 8 .....                         | 33 |
| Table 12 :Test case 9 table .....                    | 34 |
| Table 13 : Test case 10 table .....                  | 35 |

## Table of Figures

|                                             |    |
|---------------------------------------------|----|
| Figure 1: Python logo .....                 | 3  |
| Figure 2 : Visual Studio Code .....         | 4  |
| Figure 3 : FLOWCHART of the program .....   | 13 |
| Figure 4 : Sale Module Flowchart.....       | 14 |
| Figure 5 :Restock Module Flowchart .....    | 15 |
| Figure 6 : list.....                        | 19 |
| Figure 7 : Dictionary .....                 | 20 |
| Figure 8 : Formatted String example .....   | 21 |
| Figure 9 : Exception handling example ..... | 23 |
| Figure 10 : User Interface of program ..... | 24 |
| Figure 11 : Test case 1 figure.....         | 25 |
| Figure 12 : Test case 2 figure .....        | 26 |
| Figure 13 : Test case 3 figure .....        | 27 |
| Figure 14 : Test case 4 figure .....        | 28 |
| Figure 15 : Test case 5 figure .....        | 30 |
| Figure 16 : Test case 6 .....               | 31 |
| Figure 17 :Test case 7 figure .....         | 32 |
| Figure 18 : Test case 8 figure .....        | 33 |
| Figure 19 :Test case 9 before figure .....  | 34 |
| Figure 20 : Test case 9 after figure .....  | 34 |
| Figure 21 : Test case 10 figure .....       | 36 |

# **1.Introduction**

## **1.1 Project overview**

MedStore Medicine Management System is a robust and modular software solution designed to streamline pharmacy stock and sales management. This software, structured as an elegant multi-file system, separates major program functionalities into logical modules for managing stock, smart restocking, and payments management. A sophisticated sales engine, utilizing a two-level pricing scheme for individual pills and full strips, allows for seamless purchase and billing. In addition to this, MedStore has implemented an automated sales discount scheme of 5% on bulk purchases to enhance savings for customers while removing manual discount-calculation errors.

As well as managing transactions, MedStore's functionality guarantees the complete process's traceability and accountability by generating unique text invoices and implementing data persistence to always deliver up to date stock counts for use throughout the application.

The inclusion of a simple menu-driven interface and smart error-handling code contributes to efficient and smooth work experience. Consequently, MedStore Medicine Management System is undoubtedly an efficient, reliable and innovative tool to managing a pharmacy.

## **1.2 Main Purpose**

- Trace and record current medicine
- Process customer sales and generate automatic invoices
- Record restocks events and generates notes
- Update all changes in inventory.txt file

### **1.3 Aims and Objectives**

Main goal of the project is to develop a modular Command Line Interface (CLI) for automatic pharmacy inventory management system. The objectives are

- Stock Management: Manage the stock of Pharmacy and overall details
- Transaction logic: To process multi-item sales accurately and apply discount 5% when selling over two strips.
- Develop a dynamic environment for real time stock management and updating inventory.txt

## 1.4 Technologies Used

### 1.4.1 Python

Python is really a very complicated object-oriented programming language, and it is so because it is so easy to comprehend. It is because of the simplicity of Python. Python programming language was actually created by an individual named Guido Van Rossum. This was in the year 1991. In addition to being easy, it makes use of the English language grammar when writing codes. It is very versatile because it has many applications.



*Figure 1: Python logo*



### 1.4.2 VS code

Such an ultralight and efficient source code editor is called Visual Studio Code. The product was introduced into the market by Microsoft Corporation in 2015 and became one more milestone in software development tools evolution. As far as Visual Studio Code has managed to combine the advantages of both simple and efficient text editors and powerful IDEs, the distinctive feature of this tool is the availability of its modification through the use of various plugins that makes it applicable to the use with any programming languages.

In addition to the possibility of debugging using such functions as automatic code completion based on IntelliSense technology, Visual Studio Code enables using Git version control system available to users from the left sidebar. Since this free software does not depend on a certain platform and is not expensive for use, it got millions of fans around the globe. It is greatly appreciated by software developers because of its effectiveness; however, such functions as containerization and SSH remote editing may take time to get used to them.



*Figure 2 : Visual Studio Code*

## 2 Algorithm

Algorithms are defined as the step taking in solving a problem. It is essential to determine what step to step into next after another (Nikolopoulou, 2023).

### 2.1 Algorithm 1: Whole program base

1. START
2. OPEN inventory.txt
3. SPLIT each line by comma
4. APPEND each dictionary to medicine\_list
5. DISPLAY main menu
6. ACCEPT user choice
7. IF CHOICE = 1 (View Stock)  
    PRINT formatted medicine table
8. IF CHOICE = 2 (Sell Medicine)  
    WHILE TRUE  
        ACCEPT customer name  
        VALIDATE Id range and stock availability  
        CALCULATE subtotal separating strip or tablet rates  
        IF strips brought are greater than equivalent to two APPLY 5% discount  
        DEDUCT tablets from stock  
        ASK to ADD another medicine yes or no IF no, BREAK  
  
    GENERATE AND PRINT invoices to invoices  
  
    UPDATE inventory.txt  
  
    RETURN to menu
9. IF CHOICE = 3(Restock)  
    ACCEPT vendor or supplier name  
    DISPLAY current stock

VALIDATE id and quantity

ADD quantity to stock

PRINT and save restock notes to restock\_notes

RETURN to menu

10. IF CHOICE = 4(Exit)

PRINT exit message

11. ELSE

PRINT ERROR input choice is invalid

12. END

## **2.2 Algorithm 2: Module based algorithm**

### **2.2.1 Algorithm of choices.py**

inventory\_read()

1. OPEN inventory.txt in read mode
2. FOR each line split by comma
3. APPEND to medicine\_list
4. RETURN medicine\_list

Save\_inventory()

1. OPEN inventory.txt in write mode
2. FOR each medicine format as comma separated string then write to file

### **2.2.2 Algorithm of display.py**

display\_stock()

1. PRINT tabular header (Id, Name, Mg, Brand, Stock, Price)
2. FOR each medicine print formatted row

### **2.2.3 Algorithm of sales.py**

get\_next\_invoice\_number():

1. If invoices/ do not exist
2. Count files in invoices

sell\_medicine():

1. Accept customer name
2. LOOP:
  - Display stock
  - Convert quantity to total tablets

IF stock sufficient:

Calculate strips and loose tablets

Compute subtotal from rates

IF strips  $\geq 2$  apply 5% discount

Deduct from stock ADD to cart ADD to grand total

ELSE PRINT not enough stock

EXCEPT error message

Ask "Add another? (y/n)" if no, BREAK

3. PRINT Formatted bill

4. SAVE to invoices/CustomerName\_Invoice\_N.txt

5. CALL save\_inventory()

#### **2.2.4 Algorithm of restock.py**

get\_next\_note\_number():

1. IF restock\_notes doesnot exist create it
2. COUNT files in restock\_notes

restock\_medicine():

1. ACCEPT vendor or supplier name
2. DISPLAY current stock
3. VALIDATE id range
4. VALIDATE quatity is greater than zero
5. ADD quantity to medicine stock
6. FORMAT and print restock
7. SAVE to restock\_notes/VendorName\_Restock.txt
8. CALL save\_inventory()

### **2.2.5 Algorithm of main.py**

main():

1. CALL inventory\_read()
2. WHILE TRUE:
  - DISPLAY MENU
  - 1: display\_stock()
  - 2: sell\_medicine()
  - 3: restock\_medicine()
  - 4: exit loop
- ELSE: PRINT ERROR

### 3 Pseudocode

Pseudocode is stepping wise description of algorithm in simple English resembling code like structure (GeeksforGeeks, 2026).

#### 3.1 Pseudocode for sales.py

```
FUNCTION get_next_invoice_number(folder)
```

```
    IF NOT folder EXISTS THEN
```

```
        CREATE folder
```

```
    END IF
```

```
    RETURN 1
```

```
    files = LIST FILES IN folder
```

```
    RETURN LENGTH(files) + 1
```

```
END FUNCTION
```

```
PROCEDURE sell_medicine(inventory, save_function)
```

```
    customer_name = INPUT("Enter Customer Name: ")
```

```
    clean_name = REPLACE(customer_name, " ", "_")
```

```
    cart = EMPTY LIST
```

```
    grand_total = 0
```

```
    WHILE TRUE DO
```

```
        DISPLAY display_stock(inventory)
```

```
        TRY
```

```
            choice = INPUT("Enter Medicine ID to sell: ") - 1
```

```
            med = inventory[choice]
```

```
            unit = INPUT("Sell " + med['name'] + " by (tablet/strip): ").LOWER().STRIP()
```

```
            qty = INPUT("Enter quantity of " + unit + "s: ")
```

```
            total_tabs = qty * med['tabsperstrip'] IF unit = "strip" ELSE qty
```

```
            IF total_tabs > 2 THEN
```

```
                final_item_price = subtotal - discount
```

```
                med['stock'] = med['stock'] - total_tabs
```

```

        APPEND to cart
        grand_total = grand_total + final_item_price
        PRINT("Added " + med['name'] + " to cart.")
    ELSE
        PRINT("Not enough stock! (Available: " + med['stock'] + ")")
    END IF
EXCEPT
    PRINT("Invalid selection.")
END EXCEPT

IF INPUT("Add another item to this bill? (y/n): ").LOWER() != 'y' THEN
    BREAK
END IF
END WHILE

IF cart THEN
    DISPLAY bill
END IF
END

```



### 3.2 Pseudocode for Restock.py

FUNCTION restock\_medication(medicine\_inventory):

CALL show\_inventory(medicine\_list)

READ med\_id [cite: 131]

READ amount\_to\_increase

IF med\_id is valid AND quantity\_to\_increment > 0 THEN

WHERE ID = medication\_id.

MODIFY medicine, inventory + quantity\_to\_add

The 'restock\_note\_number' is a unique value created. A unique value is generated as restock\_note\_number.

Organize restock information for report

RECORD to "restock\_notes/VendorName\_Restock.txt"

CALL store\_inventory(medicine\_list)

Print "Stock has been updated successfully"

OTHERWISE

DISPLAY "Input data is not valid".

END IF

END

## 4 Flowchart

Diagrams where symbols and arrows are used to visualize a process or workflow is known as flowchart (Scheuer, 2025).

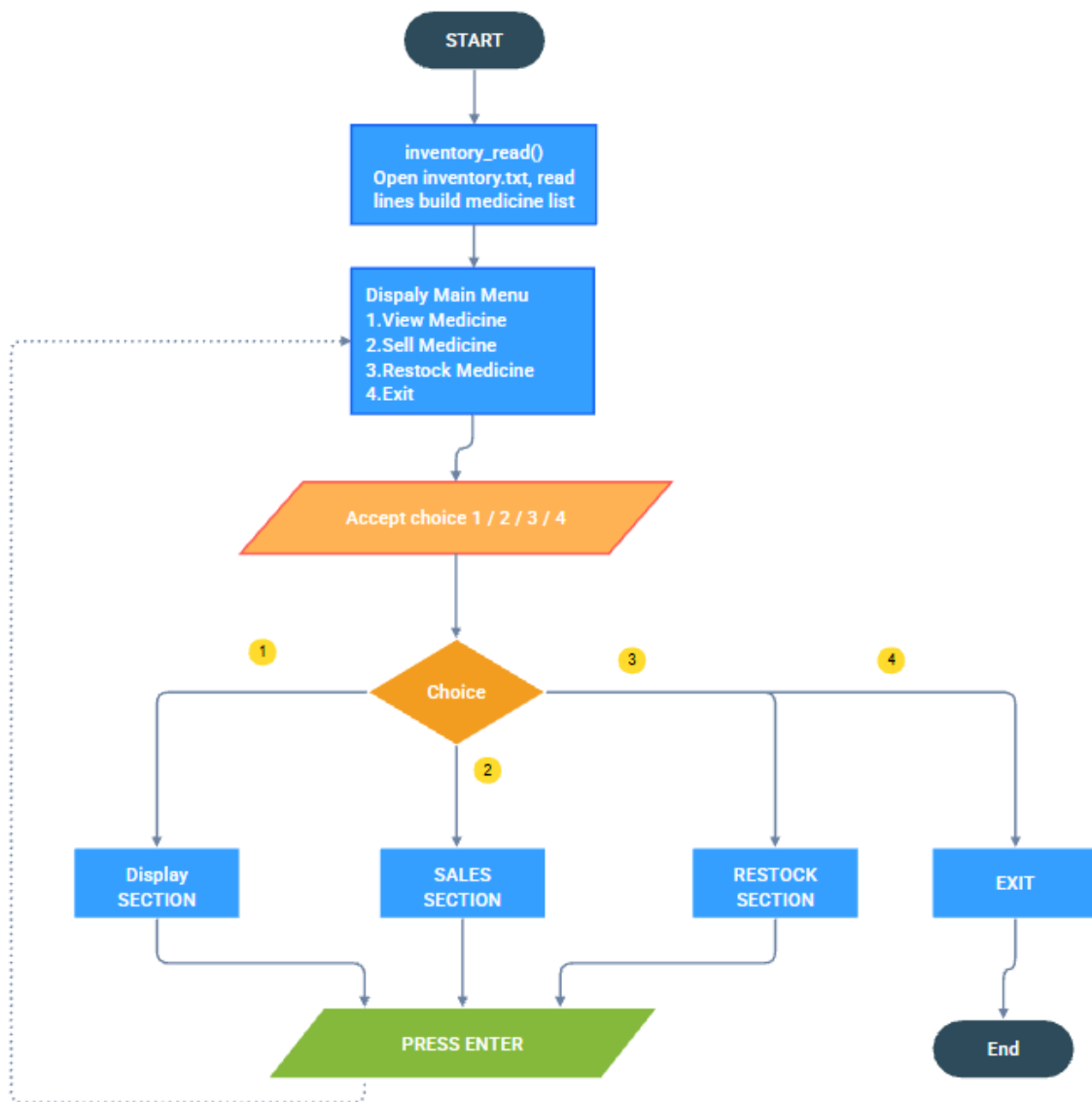


Figure 3 : FLOWCHART of the program

## 4.1 Sale Module Flowchart

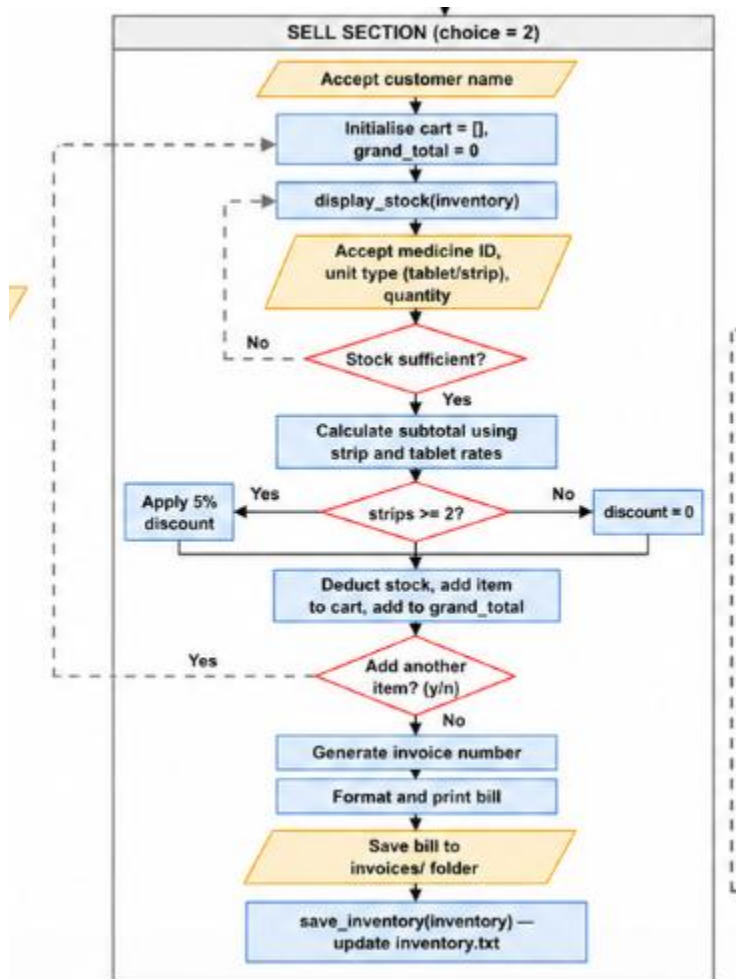


Figure 4 : Sale Module Flowchart

## 4.2 Restock Module Flowchart

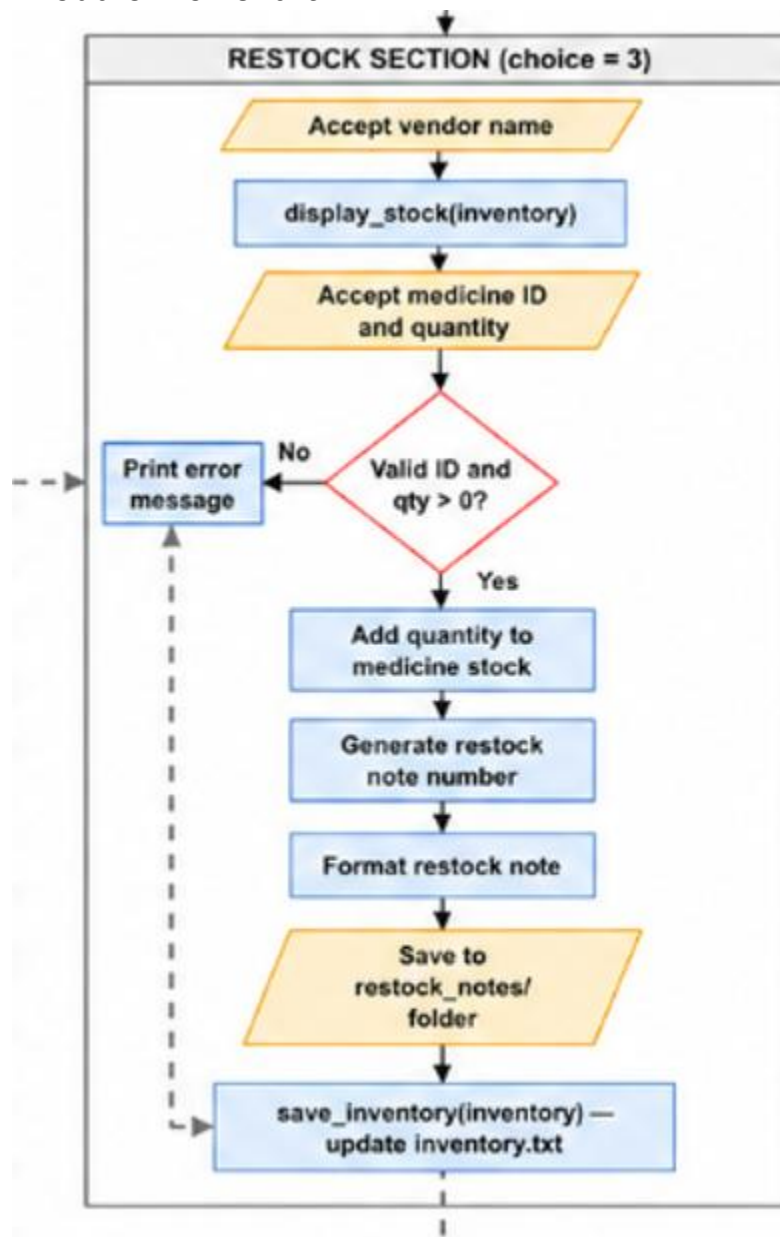


Figure 5 :Restock Module Flowchart

## 5 Data Structures

Data structures are the components needed to arrange, store, and manipulate data in a program. These can be generally categorized into two types: primitive types and collection types.

### 5.1 Types of Data Structures

#### 5.1.1 Primitive Data Structures

Primitive data structures are used to store one simple value. They are the simplest way of data representation in Python.

| Type            | Description                                                         | Example from MedStore                        |
|-----------------|---------------------------------------------------------------------|----------------------------------------------|
| Int (Integer)   | Whole number with no decimal point. Used for counting and indexing. | stock = 1740 (number of tablets in stock)    |
| Float (Decimal) | Number with a decimal point. Used for prices and rates.             | rate_tab = 5.0 (price per tablet)            |
| Str (String)    | Sequence of characters enclosed in quotes. Used for text.           | name = 'Paracetamol' (medicine name)         |
| Bool (Boolean)  | Stores either True or False. Used in condition checks.              | qty_to_add <= 0 → evaluates to True or False |

*Table 1 : Primitive Data Structures*

### 5.1.2 Collection Data Structures

Collection data structures group multiple values together into a single variable. Python provides several built-in collection types:

| Type           | Ordered?          | Mutable? | Duplicates? | Used in MedStore?               |
|----------------|-------------------|----------|-------------|---------------------------------|
| List [ ]       | Yes               | Yes      | Yes         | Yes — medicine_list, cart       |
| Dictionary { } | Yes (Python 3.7+) | Yes      | Keys: No    | Yes — medicine_info, cart items |
| Tuple ( )      | Yes               | No       | Yes         | No — data must be editable      |
| Set { }        | No                | Yes      | No          | No — order matters here         |

*Table 2 : Collection Data Structures*

## 5.2 Data Structures Used in This Program

The MedStore program uses three core data structures: List, Dictionary, and String. Each was chosen deliberately to suit the way the data is used throughout the program.

| Data Structure | Variable Name                            | File Used in            | Purpose                                                    |
|----------------|------------------------------------------|-------------------------|------------------------------------------------------------|
| List           | Medicine_py                              | Choice.py               | Stores all medicine records loaded from inventory.txt      |
| List           | Cart                                     | Sales.py                | Holds items added during one sale transaction              |
| Dictionary     | Medicine_info                            | Choice.py               | Stores all fields of one medicine as key-value pairs       |
| Dictionary     | Cart item                                | Sales.py                | Stores name, qty, unit, discount, total for each sold item |
| String         | Filename,<br>bill_output,<br>file_output | Sales.py,<br>restock.py | Builds invoice text, restock notes, and file paths         |

*Table 3 : Data Structures Used in This Program*

## 5.3 Explanation of Each Data Structure

### 5.3.1 List

#### I. Definition

#### II. Why it is used in MedStore

Two lists are used in this program:

- `medicine_list` (`choices.py`) — When `inventory.txt` is read, each line becomes one medicine dictionary. All these dictionaries are collected into a list so the program can loop through every medicine, display them in a table, and look one up by its ID.
- `cart` (`sales.py`) — During a sale, the user can add multiple medicines to one bill. A list is used because items are added one by one using `.append()` and then iterated at billing time to print each line of the invoice.

#### III. Example from Choices.py

```
#medicine ko dictionary
medicine_info={
    "name":med[0],
    "miligram":med[1],
    "brand":med[2],
    "stock":int(med[3]),
    "rate(tab)":float(med[4]),
    "rate(strip)":float(med[5]),
    "tabsperstrip":int(med[6])
}
```

Figure 6 : list



### 5.3.2 Dictionary

- I. Definition
- II. Why it is used in MedStore

Medicine table is made up of seven rows, including 'name,' 'mg,' 'brand,' 'stock,' 'cost per tab,' 'cost per strip,' and 'number of tabs per strip.' As we have used these names for naming the rows, it becomes easy for us to access them through `med['stock']` instead of `med[3]`, etc. We may say that the program subtracts total tabs from `med['stock']`.

- III. Example from Choices.py

```
#medicine ko dictionary
medicine_info={
    "name":med[0],
    "miligram":med[1],
    "brand":med[2],
    "stock":int(med[3]),
    "rate(tab)":float(med[4]),
    "rate(strip)":float(med[5]),
    "tabsperstrip":int(med[6])
}
```

*Figure 7 : Dictionary*

### 5.3.3 String

#### I. Definition

#### II. Why it is used in MedStore

Another type of data which will be utilized for this software will be strings. The applications of strings are listed below:

- Names of the drugs, names of the sellers, names of the buyers will be saved as string data that have been obtained from user inputs as well as text files.
- Formatting of the outputs can be done using strings. Sales.py and restock.py file will be converted into one string and then displayed on the screen using f-string and format().
- String data can also be used to assign names to text files containing invoices and restock notes.

#### III. Example from sales.py

```
#display in formatted way
bill_output = f"\n{'='*45}\n"
bill_output += f"{'MEDSTORE INVOICE #' + str(inv_no):^45}\n"
bill_output += f"{'='*45}\n"
bill_output += f"Customer: {customer_name}\n"
bill_output += f"{'-'*45}\n"
bill_output += f"{'Item':<20} {'Qty':<10} {'Total':<10}\n"
bill_output += f"{'-'*45}\n"
```

Figure 8 : Formatted String example

## 6 Program

### 6.1 Program explanation

Since I wanted the system to be easy to manage and update I decided to split the code into five different and manageable python files rather than one large script. Imagine that there is a `main.py` file that serves as the front door of the main menu, and `choices.py` that reads and saves our text-based inventory. The actual crux lies at the sales and restock files: `sales.py` processes the shopping cart, automatically calculates the 5% bulk discount, and adjusts the stock, while `restock.py` processes new shipments and verifies details about the supplier. Lastly, inventory tables will look clean and readable on the screen with the help of `display.py`. This way the project is organized and if something is to be fixed in the future or if something more is to be added, it can be found very easily exactly where to put it.

### 6.2 Source Code

Think of the source code as a very literal kitchen assistant's master recipe from a master chef (the programmer). The chef programs them in a human-readable and understandable language (Python or C++) but the kitchen assistant's programming language has only 1s and 0s, a robotic language. Because of this language barrier, there must be a "translator" (also known as a compiler or interpreter) who translates the recipe written in human-readable text into the form which the assistant requires to work with—namely 1s and 0s. After all, source code is the invisible plain-English coding that brings all apps, websites and games into existence—and helps make sure they work as they should.

[Source Code is in the appendix]

### 6.3 Exception Handling

Imagine exception handling as shock absorbers in your software. Typically, if someone in the app types of letters when it's asking for their age, it will freak out, freeze and crash in an instant. To avoid that, a smart safeguard has been invented by programmers and is known as "try and except". The strategy is to essentially instruct the computer to "try" to perform its regular task, but if it encounters a "snag" (error), the "except" will seize the error before the entire system crashes. Rather than crashing, and having a technical glitch, the program remains cool, explains what went wrong and helps the user correct it and continues to run smoothly.

- The Exception Handling can be seen in the image below:

```

display.display_stock(inventory)
try:
    choice = int(input("\nEnter Medicine ID to sell: ")) - 1
    med = inventory[choice]

    unit = input(f"Sell {med['name']} by (tablet/strip): ").lower().strip()
    qty = int(input(f"Enter quantity of {unit}s: "))

    total_tabs = qty * med['tabsperstrip'] if unit == "strip" else qty

    if total_tabs <= med['stock']:
        # Calculate strips and tabs for pricing
        if unit == "tablet":
            num_strips = qty // med['tabsperstrip']
            num_tabs = qty % med['tabsperstrip']
        else:
            num_strips = qty
            num_tabs = 0

        subtotal = (num_strips * med['rate(strip)']) + (num_tabs * med['rate(tab)'])

        # 5% Discount logic (if 2 or more strips are bought)
        discount = subtotal * 0.05 if num_strips >= 2 else 0
        final_item_price = subtotal - discount

        med['stock'] = med['stock'] - total_tabs
        cart.append({
            "name": med['name'],
            "qty": qty,
            "unit": unit,
            "discount": discount,
            "total": final_item_price
        })
        grand_total = grand_total + final_item_price
        print(f"Added {med['name']} to cart.")
    else:
        print(f"Not enough stock! (Available: {med['stock']})")
except (ValueError, IndexError):
    print(" Invalid selection.")

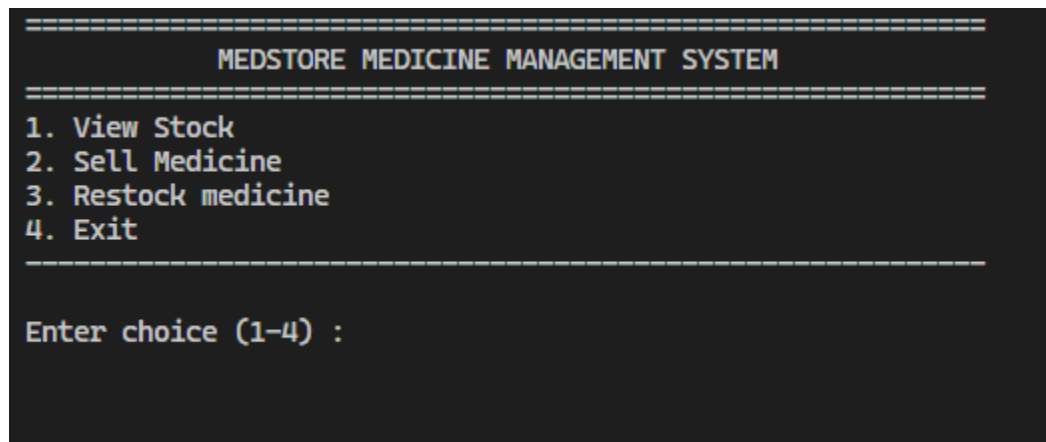
```

Figure 9 : Exception handling example

## 6.4 User Interface

Imagine a user interface (UI) as the dashboard and steering wheel of your car. The visual and functional element of the software which you see and interact with, such as a nice screen with buttons or a plain menu with typed in options. In essence, it can be regarded as a friendly interpreter that converses the language between you and the computer. When you press a button or enter a number, the UI sends simple instructions to the machine which interprets it and translates into the more complex code that the machine needs to carry out the instructions. A really excellent interface will enable you to use the software without having to put yourself into trouble reading confusing and complicated programming, and you will be able to get the job done without any problems.

- The User Interface can be seen in the image below:



*Figure 10 : User Interface of program*

## 7 Testing

### 7.1 Test Case 1: View medicine from inventory

|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| Test ID       | Test 1                                                                                  |
| Target        | Medicine list from inventory                                                            |
| Action        | <ul style="list-style-type: none"><li>• Run main.py</li><li>• Enter 1 at menu</li></ul> |
| Output        | The system displays a formatted table of medicines loaded from inventory.txt.           |
| Actual Result | The test was successful                                                                 |

Table 4 : Test case 1 table

```
PS C:\Users\asus\Desktop\foc coursework> & C:/Users/asus/AppData/Local/Python/pythoncore-3.14-64/pyth

=====
MEDSTORE MEDICINE MANAGEMENT SYSTEM
=====
1. View Stock
2. Sell Medicine
3. Restock medicine
4. Exit
=====

Enter choice (1-4) : 1

=====
| Id | Name           | Mg   | Brand           | Stock | Price(Tab) | Price(Strip) |
=====
| 1  | Paracetamol    | 500mg | Lomus           | 1740  | 5.00       | 45.00        |
| 2  | Cetirizine     | 10mg  | Quest           | 720   | 4.00       | 35.00        |
| 3  | Amoxicillin    | 500mg | Nepal Remedies  | 440   | 12.00      | 110.00       |
| 4  | Pantoprazole   | 40mg  | Deurali-Janta  | 592   | 7.00       | 60.00        |
=====

Current stock press ENTER to continue
```

Figure 11 : Test case 1 figure

## 7.2 Test Case 2: Main menu Validation

|               |                                                                                             |
|---------------|---------------------------------------------------------------------------------------------|
| Test ID       | Test 2                                                                                      |
| Target        | Menu Validation                                                                             |
| Action        | <ul style="list-style-type: none"><li>• Run main.py</li><li>• Enter invalid input</li></ul> |
| Output        | Displays error message "ENTER 1/2/3 or 4 for execution of the program "                     |
| Actual Output | The test was successful                                                                     |

Table 5 :Test case 2 table

```
=====
MEDSTORE MEDICINE MANAGEMENT SYSTEM
=====
1. View Stock
2. Sell Medicine
3. Restock medicine
4. Exit
-----

Enter choice (1-4) : 5
ENTER 1/2/3 or 4 for execution of the program
Press Enter to try again...█
```

Figure 12 : Test case 2 figure

### 7.3 Test Case 3: Exception Handling (Invalid Input)

|               |                                                                                                                                |
|---------------|--------------------------------------------------------------------------------------------------------------------------------|
| Test ID       | Test 3                                                                                                                         |
| Target        | Exception Handling                                                                                                             |
| Action        | <ul style="list-style-type: none"><li>• Run main.py</li><li>• Enter 2 on the menu</li><li>• Enter 'ten' for quantity</li></ul> |
| Output        | System encounters invalid id and display message invalid selection                                                             |
| Actual Output | The test was successful                                                                                                        |

Table 6 : Test case 3

```
=====
MEDSTORE MEDICINE MANAGEMENT SYSTEM
=====
1. View Stock
2. Sell Medicine
3. Restock medicine
4. Exit
=====

Enter choice (1-4) : 2
Enter Customer Name: test

-----
| Id | Name           | Mg   | Brand           | Stock | Price(Tab) | Price(Strip) |
-----
| 1  | Paracetamol    | 500mg | Lomus           | 1740  | 5.00       | 45.00        |
| 2  | Cetirizine     | 10mg  | Quest           | 720   | 4.00       | 35.00        |
| 3  | Amoxicillin    | 500mg | Nepal Remedies  | 440   | 12.00      | 110.00       |
| 4  | Pantoprazole   | 40mg  | Deurali-Janta  | 592   | 7.00       | 60.00        |
-----

Current stock press ENTER to continue

Enter Medicine ID to sell: 10
Invalid selection.
```

Figure 13 : Test case 3 figure



## 7.4 Test Case 4: Stock Validity check

|               |                                                 |
|---------------|-------------------------------------------------|
| Test ID       | Test 4                                          |
| Target        | Stock Validation                                |
| Action        | Attempt to buy stock greater than current stock |
| Output        | System displays 'Not enough stock!' message.    |
| Actual Output | Test was successful                             |

Table 7 : Test case 4 table

```
=====
MEDSTORE MEDICINE MANAGEMENT SYSTEM
=====
1. View Stock
2. Sell Medicine
3. Restock medicine
4. Exit
=====

Enter choice (1-4) : 2
Enter Customer Name: test

=====
| Id | Name           | Mg   | Brand           | Stock | Price(Tab) | Price(Strip) |
=====
| 1  | Paracetamol    | 500mg | Lomus           | 1740  | 5.00       | 45.00        |
| 2  | Cetirizine     | 10mg  | Quest           | 720   | 4.00       | 35.00        |
| 3  | Amoxicillin    | 500mg | Nepal Remedies  | 440   | 12.00      | 110.00       |
| 4  | Pantoprazole   | 40mg  | Deurali-Janta  | 592   | 7.00       | 60.00        |
=====

Current stock press ENTER to continue

Enter Medicine ID to sell: 1
Sell Paracetamol by (tablet/strip): tablet
Enter quantity of tablets: 300000
Not enough stock! (Available: 1740)

Add another item to this bill? (y/n): █
=====
```

Figure 14 : Test case 4 figure

## 7.5 Test Case 5: Multi item Processing and Discount Calculation

|               |                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------|
| Test ID       | Test 5                                                                                                                |
| Target        | Multiple Sales & Discount                                                                                             |
| Action        | <ul style="list-style-type: none"><li>• Add multiple items to cart</li><li>• Buy <math>\geq 2</math> strips</li></ul> |
| Output        | Items added to cart<br>5% discount applied correctly to subtotals.                                                    |
| Actual Output | The test was successfull                                                                                              |

*Table 8: Test case 5*

```

Sett Amoxicillin by (tablet/strip): strip
Enter quantity of strips: 3
Added Amoxicillin to cart.

Add another item to this bill? (y/n): n

=====
MEDSTORE INVOICE #2
=====
Customer: ester
-----
Item                Qty      Disc      Total
-----
Paracetamol         5 strip Rs.11.25  Rs.  213.75
Cetirizine          5 tablet Rs. 0.00   Rs.   20.00
Amoxicillin         3 strip Rs.16.50   Rs.  313.50
-----
GRAND TOTAL:                Rs.  547.25
=====

Invoice saved as: invoices/ester_Invoice_2.txt

```

Figure 15 : Test case 5 figure

## 7.6 Test Case 6: Automatic invoice generator

|               |                                                             |
|---------------|-------------------------------------------------------------|
| Test ID       | Test 6                                                      |
| Target        | Unique File Creation (Sales)                                |
| Action        | Complete sale for customer 'Ram'                            |
| Output        | File 'Ram_Invoice_1.txt' is created in the invoices folder. |
| Actual Output | The test was sucessful                                      |

Table 9 : Test case 6 table

```
Current stock press ENTER to continue

Enter Medicine ID to sell: 4
Sell Pantoprazole by (tablet/strip): tablet
Enter quantity of tablets: 5
Added Pantoprazole to cart.

Add another item to this bill? (y/n): n

=====
MEDSTORE INVOICE #3
=====
Customer: ram
=====
Item          Qty      Disc      Total
=====
Pantoprazole   5 tablet Rs. 0.00  Rs. 35.00
=====
GRAND TOTAL:                Rs. 35.00
=====

Success! Invoice saved in 'invoices' folder as: invoices/ram_Invoice_3.txt
```

Figure 16 : Test case 6

## 7.7 Test Case 7: Medicine range and Quantity range validation

|               |                                                             |
|---------------|-------------------------------------------------------------|
| Test ID       | Test 7                                                      |
| Target        | Input Validation                                            |
| Action        | In Restock, enter invalid ID (e.g. 99) or negative quantity |
| Output        | Displays invalid error "Restock cancel "                    |
| Actual output | Test case was successfull                                   |

Table 10 : Test case 7

```
Enter choice (1-4) : 3
Enter Vendor/Supplier Name: ramu

-----
| Id | Name           | Mg   | Brand           | Stock | Price(Tab) | Price(Strip) |
-----
| 1  | Paracetamol   | 500mg | Lomus           | 1660  | 5.00       | 45.00        |
| 2  | Cetirizine    | 10mg  | Quest           | 655   | 4.00       | 35.00        |
| 3  | Amoxicillin   | 500mg | Nepal Remedies  | 390   | 12.00      | 110.00       |
| 4  | Pantoprazole  | 40mg  | Deurali-Janta  | 585   | 7.00       | 60.00        |
-----

Current stock press ENTER to continue

Enter Medicine ID to restock: 99
Invalid ID. Restock cancelled.
```

Figure 17 :Test case 7 figure

## 7.8 Test Case 8: Unique Restock Note Generation

|               |                                                                   |
|---------------|-------------------------------------------------------------------|
| Test ID       | Test 8                                                            |
| Feature       | Unique File Creation (Restock)                                    |
| Action        | Complete restock for vendor 'HealthCo'                            |
| Output        | File 'HealthCo_Restock_1.txt' is created in restock_notes folder. |
| Actual Output | The Test was successful                                           |

Table 11 : Test case 8

```
Current stock press ENTER to continue

Enter Medicine ID to restock: 1
Enter quantity of tablets to add: 10

=====
|          RESTOCK CONFIRMATION          |
=====
| Description      | Entry      |
|-----|-----|
| Vendor          | HealthCo   |
| Medicine        | Paracetamol |
| Quantity Added  | 10         |
| New Total Stock | 1670       |
|-----|-----|
=====

Success: Restock note saved as restock_notes/HealthCo_Restock_3.txt
```

Figure 18 : Test case 8 figure

## 7.9 Test Case 9: Dynamic file update inventory.txt

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| Test ID       | Test 9                                                                     |
| Target        | Data File Update                                                           |
| Action        | Finalize transaction and open inventory.txt                                |
| Output        | inventory.txt correctly reflects updated stock values after sales/restock. |
| Actual Output | The test was successful                                                    |

Table 12 :Test case 9 table

```
Paracetamol, 500mg, Lomus,1670,5.0,45.0,10
Cetirizine, 10mg, Quest,655,4.0,35.0,12
Amoxicillin, 500mg, Nepal Remedies,390,12.0,110.0,15
Pantoprazole, 40mg, Deurali-Janta,585,7.0,60.0,8
```

Figure 19 :Test case 9 before figure

```
inventory.txt
1  Paracetamol, 500mg, Lomus,2670,5.0,45.0,10
2  Cetirizine, 10mg, Quest,655,4.0,35.0,12
3  Amoxicillin, 500mg, Nepal Remedies,390,12.0,110.0,15
4  Pantoprazole, 40mg, Deurali-Janta,585,7.0,60.0,8
5
```

Figure 20 : Test case 9 after figure

## 7.10 Test Case 10: Inventory Accuracy

|               |                                                                              |
|---------------|------------------------------------------------------------------------------|
| Test ID       | Test 10                                                                      |
| Feature       | Inventory Accuracy                                                           |
| Action        | Buy the exact remaining amount of a medicine (e.g., stock is 10, buy 10).    |
| Output        | Transaction succeeds; inventory.txt correctly shows '0' stock for that item. |
| Actual Result | The test was successful                                                      |

*Table 13 : Test case 10 table*



```
| Id | Name | Mg | Brand | Stock | Price(Tab) | Price(Strip) |
| 1 | Paracetamol | 500mg | Lomus | 2670 | 5.00 | 45.00 |
| 2 | Cetirizine | 10mg | Quest | 655 | 4.00 | 35.00 |
| 3 | Amoxicillin | 500mg | Nepal Remedies | 390 | 12.00 | 110.00 |
| 4 | Pantoprazole | 40mg | Deurali-Janta | 585 | 7.00 | 60.00 |

Current stock press ENTER to continue

Enter Medicine ID to sell: 1
Sell Paracetamol by (tablet/strip): tablet
Enter quantity of tablets: 2670
Added Paracetamol to cart.

Add another item to this bill? (y/n): n

=====
MEDSTORE INVOICE #6
=====
Customer: raju
=====
Item Qty Disc Total
-----
Paracetamol 2670 tablet Rs. 0.00 Rs.12015.00
-----
GRAND TOTAL: Rs.12015.00
=====

Success! Invoice saved in 'invoices' folder as: invoices/raju_Invoice_6.txt

=====
MEDSTORE MEDICINE MANAGEMENT SYSTEM
=====
1. View Stock
2. Sell Medicine
3. Restock medicine
4. Exit
=====

Enter choice (1-4) : 1

| Id | Name | Mg | Brand | Stock | Price(Tab) | Price(Strip) |
| 1 | Paracetamol | 500mg | Lomus | 0 | 5.00 | 45.00 |
| 2 | Cetirizine | 10mg | Quest | 655 | 4.00 | 35.00 |
| 3 | Amoxicillin | 500mg | Nepal Remedies | 390 | 12.00 | 110.00 |
| 4 | Pantoprazole | 40mg | Deurali-Janta | 585 | 7.00 | 60.00 |

Current stock press ENTER to continue
```

Figure 21 : Test case 10 figure

## 8 Conclusion

The MedStore Medicine Management System has been successfully developed and delivers a powerful and easy to use command-line system that could be used in small-scale pharmaceutical inventory management. During the entire project life cycle, several important computational concepts were adopted to make the system reliable and efficient.

### Project Achievements:

**Data Persistence:** It also includes a text-based database (inventory.txt), which ensures that all the stock and pricing data are stored for multiple sessions, avoiding data loss.

**Advanced Logic:** The program's ability to deal with complex business rules, which are not limited to just data entry, is showcased through the implementation of dynamic pricing, in this case, the 5% bulk discount for strip-based purchases.

**Automated Documentation:** It automatically creates individual, dated invoices and restock notes, which are neatly kept in separate sub-folders (invoices/ and restock\_notes/) to keep track of the documents.

**Error Resilience:** There are a number of try-except blocks in this program, plus some checking of inputs, so that the program will not be destabilized by unexpected user actions such as entering non-numeric data or making an out-of-stock request.

### Final Reflection:

This is a project in which the importance of interface and usability is emphasized, while at the same time the back-end code is developed. This is because the basic program is written using advanced dictionary manipulation and I/O techniques, yet the last user interface is sufficiently user-friendly for a non-technical user to use. All 10 cases tested successfully, which has shown the system is able to meet all the initial requirements and it will lay a foundation for further development of the system, such as the introduction of a graphical user interface (GUI) and an integrated system for managing the expiry date.

## 9 Appendix

### 9.1 Main.py

```
'''
Fundamentals of Computing coursework,
Medicine management system
'''

#imports from seperate files
import choices
import display
import sales
import restock
#main menu
def main():
    #load the medicine to the program
    inventory=choices.inventory_read()

    while True:
        print("\n"+"="*60)
        print(f'{"MEDSTORE MEDICINE MANAGEMENT SYSTEM":^60}')
        print("="*60)
        print("1. View Stock")
        print("2. Sell Medicine")
        print("3. Restock medicine")
        print("4. Exit")
        print("-"* 60)
        print(" ")
        choice = input("Enter choice (1-4) : ")

        # choice 1 - see what is in the stock
```

```

if choice=="1":
    #calling display function to display inventory
    display.display_stock(inventory)

elif choice == "2":
    #calling function to sell medicine
    sales.sell_medicine(inventory,choices.save_inventory)

elif choice == "3":
    #calling function to restock medicine
    restock.restock_medicine(inventory,choices.save_inventory)

elif choice=="4":
    print("closing system Medstore Management system")
    break

#terminate wrong input
else:
    print("ENTER 1/2/3 or 4 for execution of the program ")
    #prevent from execution directly
    input("Press Enter to try again...")

# Boiler-plate code
if __name__=="__main__":
    main()

```

## 9.2 Restock.py

```
import os
import display

def get_next_note_number(folder="restock_notes"):
    """
    Calculates number of files in the restock_notes directory to generate the next unique
    id.
    """
    if os.path.exists(folder) == False:
        return 1

    files = os.listdir(folder)
    next_number = len(files) + 1
    return next_number

def restock_medicine(inventory, save_function):
    """
    Processes restocking procedure, updates the memory and outputs the confirmation
    table in proper format.
    """
    vendor_name = input("Enter Vendor/Supplier Name: ").strip()

    #Rename filename manually (without using .replace function shortcut)
    clean_vendor = ""
    for char in vendor_name:
        if char == " ":
            clean_vendor = clean_vendor + "_"
        else:
            clean_vendor = clean_vendor + char
```

```

# Display current stock for user reference
display.display_stock(inventory)

try:
    # Get ID and convert to index
    user_input_id = input("\nEnter Medicine ID to restock: ")
    choice = int(user_input_id) - 1

    # Validation for ID range
    if choice < 0 or choice >= len(inventory):
        print("Invalid ID. Restock cancelled.")
        return

    # Get Quantity of restocking items
    input_qty = input("Enter quantity of tablets to add: ")
    qty_to_add = int(input_qty)

    #if quantity is less then zero show the error
    if qty_to_add <= 0:
        print("Quantity must be greater than zero.")
        return

    old_stock = inventory[choice]['stock']
    new_stock = old_stock + qty_to_add
    inventory[choice]['stock'] = new_stock

    # Determine file path
    note_no = get_next_note_number("restock_notes")
    filename = "restock_notes/" + clean_vendor + "_Restock_" + str(note_no) + ".txt"

```

```

# formatted tabular format display for output
border = "===== "

# for proper alignment and clean UI i have used .format function
header_row = "| {:<20} | {:<18} |".format("Description", "Entry")
separator = "|-----|-----|"

# Print and update dynamically in file and output screen
final_output = "\n" + border + "\n"
final_output = final_output + "|{: ^43}|".format("RESTOCK CONFIRMATION") + "\n"
final_output = final_output + border + "\n"
final_output = final_output + header_row + "\n"
final_output = final_output + separator + "\n"
final_output = final_output + "| {:<20} | {:<18} |".format("Vendor", vendor_name) +
"\n"
        final_output = final_output + "| {:<20} | {:<18} |".format("Medicine",
inventory[choice]['name']) + "\n"
        final_output = final_output + "| {:<20} | {:<18} |".format("Quantity Added",
str(qty_to_add)) + "\n"
        final_output = final_output + "| {:<20} | {:<18} |".format("New Total Stock",
str(inventory[choice]['stock'])) + "\n"
        final_output = final_output + border + "\n"

# Display to User
print(final_output)

# file not found exception handling
try:
    with open(filename, "w") as f:
        f.write(final_output)
        print("Success: Restock note saved as " + filename)
except FileNotFoundError:

```

```

        print(" Folder 'restock_notes' not found. File was not saved.")

# Update the master inventory.txt file
save_function(inventory)

except ValueError:
    print("Please enter numerical values for ID and Quantity.")

```

### 9.3 Sales.py

```

import os
import display

def get_next_invoice_number(folder="invoices"):
    # Create the folder if it doesn't exist
    if not os.path.exists(folder):
        os.makedirs(folder)
    return 1
    files = os.listdir(folder)
    return len(files) + 1

def sell_medicine(inventory, save_function):
    customer_name = input("Enter Customer Name: ").strip()
    clean_name = customer_name.replace(" ", "_")
    cart = []
    grand_total = 0

    while True:
        display.display_stock(inventory)
        try:
            choice = int(input("\nEnter Medicine ID to sell: ")) - 1

```



```

if choice < 0 or choice >= len(inventory):
    print("Invalid ID selection.")
    continue

med = inventory[choice]

unit = input(f"Sell {med['name']} by (tablet/strip): ").lower().strip()
qty = int(input(f"Enter quantity of {unit}s: "))

total_tabs = qty * med['tabsperstrip'] if unit == "strip" else qty

if total_tabs <= med['stock']:
    if unit == "tablet":
        num_strips = qty // med['tabsperstrip']
        num_tabs = qty % med['tabsperstrip']
    else:
        num_strips = qty
        num_tabs = 0

    subtotal = (num_strips * med['rate(strip)']) + (num_tabs * med['rate(tab)'])

    # Apply 5% discount for 2 or more strips
    discount = subtotal * 0.05 if (unit == "strip" and qty >= 2) else 0
    final_item_price = subtotal - discount

    med['stock'] -= total_tabs
    cart.append({
        "name": med['name'],
        "qty": qty,
        "unit": unit,
        "subtotal": subtotal,

```

```

        "discount": discount,
        "total": final_item_price
    })
    grand_total += final_item_price
    print(f"Added {med['name']} to cart.")
else:
    print(f"Not enough stock! (Available: {med['stock']})")
except (ValueError, IndexError):
    print("Invalid input.")

if input("\nAdd another item to this bill? (y/n): ").lower() != 'y':
    break

if cart:
    inv_no = get_next_invoice_number("invoices")

    filename = f"invoices/{clean_name}_Invoice_{inv_no}.txt"

    bill_output = f"\n{'='*45}\n"
    bill_output += f"{'MEDSTORE INVOICE #' + str(inv_no):^45}\n"
    bill_output += f"{'='*45}\n"
    bill_output += f"Customer: {customer_name}\n"
    bill_output += f"{'-'*45}\n"
    bill_output += f"{'Item':<18} {'Qty':<8} {'Disc':<8} {'Total':<10}\n"
    bill_output += f"{'-'*45}\n"

    for item in cart:
        bill_output += f"{'item['name']:<18} {item['qty']} {item['unit']:<4} "
        bill_output += f"Rs.{item['discount']:>5.2f} Rs.{item['total']:>8.2f}\n"

```



```
input("\n Current stock press ENTER to continue")
```

## 9.5 Choice.py

```
'''
```

```
Main logic of overaall systeem is maintained` here
```

```
'''
```

```
# load data from inventory.txt
```

```
def inventory_read():
```

```
    medicine_list=[] #empty list for medicine
```

```
    with open("inventory.txt","r") as file:#open file in read mode
```

```
        for line in file:
```

```
            #at a time that line variable will hold one line at once
```

```
                med=line.strip().split(",") #seperate the sentence into list of words ` , `
```

```
                #medicine ko dictionary
```

```
                medicine_info={
```

```
                    "name":med[0],
```

```
                    "miligram":med[1],
```

```
                    "brand":med[2],
```

```
                    "stock":int(med[3]),
```

```
                    "rate(tab)":float(med[4]),
```

```
                    "rate(strip)":float(med[5]),
```

```
                    "tabsperstrip":int(med[6])
```

```
                }
```

```
                medicine_list.append(medicine_info)
```

```
    return medicine_list
```

#function to edit updated data in file

```
def save_inventory(inventory):
```

```
    with open("inventory.txt","w") as file:
```

```
        for med in inventory:
```

```
            line =  
f'{med['name']},{med['miligram']},{med['brand']},{med['stock']},{med['rate(tab)']},{med['rate(strip)']},{med['tabsperstrip']}\n'  
            file.write(line)
```

## 9.6 Inventory.txt

Paracetamol, 500mg, Lomus,0,5.0,45.0,10

Cetirizine, 10mg, Quest,655,4.0,35.0,12

Amoxicillin, 500mg, Nepal Remedies,390,12.0,110.0,15

Pantoprazole, 40mg, Deurali-Janta,585,7.0,60.0,8